Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	**Slot:** L43 + L44
Regno: 23BCE1145	**Name:** S Vishwajith

EX9 – DSS

Code:

Sender:

```
import socket

import random

import sympy

import copy


buffers = [

0x6A09E667F3BCC908, 0xBB67AE8584CAA73B, 0x3C6EF372FE94F82B, 0xA54FF53A5F1D36F1,

0x510E527FADE682D1, 0x9B05688C2B3E6C1F, 0x1F83D9ABFB41BD6B, 0x5BE0CD19137E2179,

]

K = [

0x428A2F98D728AE22, 0x7137449123EF65CD, 0xB5C0FBCFEC4D3B2F, 0xE9B5DBA58189DBBC,

0x3956C25BF348B538, 0x59F111F1B605D019, 0x923F82A4AF194F9B, 0xAB1C5ED5DA6D8118,

0xD807AA98A3030242, 0x12835B0145706FBE, 0x243185BE4EE4B28C, 0x550C7DC3D5FFB4E2,

0x72BE5D74F27B896F, 0x80DEB1FE3B1696B1, 0x9BDC06A725C71235, 0xC19BF174CF692694,

0xE49B69C19EF14AD2, 0xEFBE4786384F25E3, 0x0FC19DC68B8CD5B5, 0x240CA1CC77AC9C65,

0x2DE92C6F592B0275, 0x4A7484AA6EA6E483, 0x5CB0A9DCBD41FBD4, 0x76F988DA831153B5,

0x983E5152EE66DFAB, 0xA831C66D2DB43210, 0xB00327C898FB213F, 0xBF597FC7BEEF0EE4,

0xC6E00BF33DA88FC2, 0xD5A79147930AA725, 0x06CA6351E003826F, 0x142929670A0E6E70,

0x27B70A8546D22FFC, 0x2E1B21385C26C926, 0x4D2C6DFC5AC42AED, 0x53380D139D95B3DF,

0x650A73548BAF63DE, 0x766A0ABB3C77B2A8, 0x81C2C92E47EDAEE6, 0x92722C851482353B,

0xA2BFE8A14CF10364, 0xA81A664BBC423001, 0xC24B8B70D0F89791, 0xC76C51A30654BE30,

0xD192E819D6EF5218, 0xD69906245565A910, 0xF40E35855771202A, 0x106AA07032BBD1B8,
```

```
0x19A4C116B8D2D0C8, 0x1E376C085141AB53, 0x2748774CDF8EEB99, 0x34B0BCB5E19B48A8,
0x391C0CB3C5C95A63, 0x4ED8AA4AE3418ACB, 0x5B9CCA4F7763E373, 0x682E6FF3D6B2B8A3,
0x748F82EE5DEFB2FC, 0x78A5636F43172F60, 0x84C87814A1F0AB72, 0x8CC702081A6439EC,
0x90BEFFFA23631E28, 0xA4506CEBDE82BDE9, 0xBEF9A3F7B2C67915, 0xC67178F2E372532B,
0xCA273ECEEA26619C, 0xD186B8C721C0C207, 0xEADA7DD6CDE0EB1E, 0xF57D4F7FEE6ED178,
0x06F067AA72176FBA, 0x0A637DC5A2C898A6, 0x113F9804BEF90DAE, 0x1B710B35131C471B,
0x28DB77F523047D84, 0x32CAAB7B40C72493, 0x3C9EBE0A15C9BEBC, 0x431D67C49C100D4C,
0x4CC5D4BECB3E42B6, 0x597F299CFC657E2A, 0x5FCB6FAB3AD6FAEC, 0x6C44198C4A475817,
]
state = copy.deepcopy(buffers)

def to_int(h):
    if(h=='''): return 0
    else: return int(h,16)

def num_blocks(l):
    n = (-l-1-128)%1024
    if(n==0): n=1024
    return((l+128+n+1)//1024, n)

def rotr(w,n):
    w=bin(w)[2:].rjust(64,'0')
    w=w[-n:]+w[:-n]
    return int(w,2)

def s_512(w,n):
    nums=[(1,8,7),(19,61,6)]
    x=nums[n]
    w=to_int(w)
    return (rotr(w,x[0]) ^ rotr(w,x[1]) ^ w>>x[2])
```

```
def sigma(w,n):
    nums=[(28,34,39),(14,18,41)]
    x=nums[n]
    return (rotr(w,x[0]) ^ rotr(w,x[1]) ^ rotr(w,x[2]))

def ch(e,f,g):
    return (e & f) ^ ((e ^ (2**64-1)) & g)

def maj(a,b,c):
    return (a & b) ^ (b & c) ^ (c & a)

def gen_words(msg):
    w = []
    for i in range(16):
        w.append(msg[16*i:16*i+16])
    for i in range(16,80):
        nw = ((s_512(w[i-2],1)+s_512(w[i-15],0)+to_int(w[i-16])+to_int(w[i-7]))%(2**64))
        w.append(hex(nw)[2:])
    return w

def sha(msg):
    [a,b,c,d,e,f,g,h]=state
    words = gen_words(hex(msg)[2:].rjust(256,'0'))
    for r in range(80):
        t1 = (h+ch(e,f,g)+sigma(e,1)+to_int(words[r])+K[r])%(2**64)
        t2 = (sigma(a,0)+maj(a,b,c))%(2**64)
        nh = g
        ng = f
        nf = e
        ne = (d+t1)%(2**64)
        nd = c
```

```
        nc = b
        nb = a
        na = (t1+t2)%(2**64)
        a,b,c,d,e,f,g,h = na,nb,nc,nd,ne,nf,ng,nh
    strout = "0x"
    for x in [a,b,c,d,e,f,g,h]:
        strout+=hex(x)[2:].rjust(16,'0')
    bufs = [a,b,c,d,e,f,g,h]
    strout = "0x"
    for i in range(8):
        state[i] = (state[i] + bufs[i])%(2**64)
        strout+=hex(state[i])[2:].rjust(16,'0')

plain = input("Enter message: ")
h = plain.encode('ascii').hex()
l = 4*len(h)
print(f"Input bits: {l}")
h = (to_int(h)<<1) + 1
nblocks, npadding = num_blocks(l)
h = (h<<(npadding+128))+(l)
print(f"Number of blocks: {nblocks}\n")
blocks = []
for i in range(nblocks):
    blocks.append(to_int(hex(h)[2+256*i:258+256*i].rjust(256,'0')))
for block in blocks: sha(block)
strout = "0x"
for i in range(8):
    strout+=hex(state[i])[2:].rjust(16,'0')
print(f"Final Hash: {strout}\n")
hash=int(strout,16)
s = socket.socket()
```

```
s.connect(('127.0.0.1',10000))

def generate_q(N):
    return sympy.randprime(2 ** (N - 1), 2 ** N)

def generate_p(L, q):
    lo = (1 << (L - 1)) // q
    hi = ((1 << L) - 1) // q
    while True:
        k = random.randint(lo, hi)
        if k % 2 != 0:
            k += 1
        p = k * q + 1
        if p.bit_length() == L and sympy.isprime(p):
            return p

def generate_g(p, q):
    e = (p - 1) // q
    while True:
        h = random.randint(2, p - 2)
        g = pow(h, e, p)
        if g != 1:
            return g

def generate_dsa_params(L = 1024, N = 160):
    q = generate_q(N)
    p = generate_p(L, q)
    g = generate_g(p, q)
    return p, q, g

[p,q,g]=generate_dsa_params()
```

```
print(f"P: {p}\n\nQ: {q}\n\nG: {g}\n")
x=random.randint(2,q-1)
k=random.randint(2,q-1)
print(f"Sender Private Key(X): {x}\n\nPer-message random number(K): {k}\n")
y=pow(g,x,p)
print(f"Sender Public Key(Y): {y}\n")
s.send((str(p)+","+str(q)+","+str(g)+","+str(y)).encode())
assert(s.recv(1024).decode()=="ack")
r=(pow(g,k,p))%q
sv=(pow(k,-1,q)*(hash+x*r))%q
print(f"R: {r}\n\nS: {sv}")
s.send((str(plain)+","+str(r)+","+str(sv)).encode())
```

Receiver:

```
import socket
import copy

s = socket.socket()
s.bind(('',10000))
s.listen(10)
c, addr = s.accept()
pqg = c.recv(1024).decode()
[p,q,g,y] = pqg.split(',')
[p,q,g,y] = [int(p),int(q),int(g),int(y)]
print(f"P: {p}\n\nQ: {q}\n\nG: {g}\n")
print(f"Sender Public Key(Y): {y}\n")
c.send("ack".encode())
[plain,r,s]=c.recv(1024).decode().split(',')
[plain,r,s]=[plain,int(r),int(s)]
err=input("Simulate error? Y/N: ")
if(err=="Y"): plain=plain[:3]+chr(ord(plain[3])+2)+plain[4:]
print(f"\nMessage (M): {plain}\n\nReceived R: {r}\n\nReceived S: {s}\n")
```

```
buffers = [
0x6A09E667F3BCC908, 0xBB67AE8584CAA73B, 0x3C6EF372FE94F82B, 0xA54FF53A5F1D36F1,
0x510E527FADE682D1, 0x9B05688C2B3E6C1F, 0x1F83D9ABFB41BD6B, 0x5BE0CD19137E2179
]
K = [
0x428A2F98D728AE22, 0x7137449123EF65CD, 0xB5C0FBCFEC4D3B2F, 0xE9B5DBA58189DBBC,
0x3956C25BF348B538, 0x59F111F1B605D019, 0x923F82A4AF194F9B, 0xAB1C5ED5DA6D8118,
0xD807AA98A3030242, 0x12835B0145706FBE, 0x243185BE4EE4B28C, 0x550C7DC3D5FFB4E2,
0x72BE5D74F27B896F, 0x80DEB1FE3B1696B1, 0x9BDC06A725C71235, 0xC19BF174CF692694,
0xE49B69C19EF14AD2, 0xEFBE4786384F25E3, 0x0FC19DC68B8CD5B5, 0x240CA1CC77AC9C65,
0x2DE92C6F592B0275, 0x4A7484AA6EA6E483, 0x5CB0A9DCBD41FBD4, 0x76F988DA831153B5,
0x983E5152EE66DFAB, 0xA831C66D2DB43210, 0xB00327C898FB213F, 0xBF597FC7BEEF0EE4,
0xC6E00BF33DA88FC2, 0xD5A79147930AA725, 0x06CA6351E003826F, 0x142929670A0E6E70,
0x27B70A8546D22FFC, 0x2E1B21385C26C926, 0x4D2C6DFC5AC42AED, 0x53380D139D95B3DF,
0x650A73548BAF63DE, 0x766A0ABB3C77B2A8, 0x81C2C92E47EDAEE6, 0x92722C851482353B,
0xA2BFE8A14CF10364, 0xA81A664BBC423001, 0xC24B8B70D0F89791, 0xC76C51A30654BE30,
0xD192E819D6EF5218, 0xD69906245565A910, 0xF40E35855771202A, 0x106AA07032BBD1B8,
0x19A4C116B8D2D0C8, 0x1E376C085141AB53, 0x2748774CDF8EEB99, 0x34B0BCB5E19B48A8,
0x391C0CB3C5C95A63, 0x4ED8AA4AE3418ACB, 0x5B9CCA4F7763E373, 0x682E6FF3D6B2B8A3,
0x748F82EE5DEFB2FC, 0x78A5636F43172F60, 0x84C87814A1F0AB72, 0x8CC702081A6439EC,
0x90BEFFFA23631E28, 0xA4506CEBDE82BDE9, 0xBEF9A3F7B2C67915, 0xC67178F2E372532B,
0xCA273ECEEA26619C, 0xD186B8C721C0C207, 0xEADA7DD6CDE0EB1E, 0xF57D4F7FEE6ED178,
0x06F067AA72176FBA, 0x0A637DC5A2C898A6, 0x113F9804BEF90DAE, 0x1B710B35131C471B,
0x28DB77F523047D84, 0x32CAAB7B40C72493, 0x3C9EBE0A15C9BEBC, 0x431D67C49C100D4C,
0x4CC5D4BECB3E42B6, 0x597F299CFC657E2A, 0x5FCB6FAB3AD6FAEC, 0x6C44198C4A475817,
]
state = copy.deepcopy(buffers)

def Convert_hex_to_int(h):
    if(h=='''): return 0
```

```
    else: return int(h,16)

def Number_of_blocks(l):
    n = (-l-1-128)%1024
    if(n==0): n=1024
    return((l+128+n+1)//1024, n)

def rotr(w,n):
    w=bin(w)[2:].rjust(64,'0')
    w=w[-n:]+w[:-n]
    return int(w,2)

def s_512(w,n):
    nums=[(1,8,7),(19,61,6)]
    x=nums[n]
    w=Convert_hex_to_int(w)
    return (rotr(w,x[0]) ^ rotr(w,x[1]) ^ w>>x[2])

def sigma(w,n):
    nums=[(28,34,39),(14,18,41)]
    x=nums[n]
    return (rotr(w,x[0]) ^ rotr(w,x[1]) ^ rotr(w,x[2]))

def ch(e,f,g):
    return (e & f) ^ ((e ^ (2**64-1)) & g)

def maj(a,b,c):
    return (a & b) ^ (b & c) ^ (c & a)

def Generate_words(msg):
    w = []
```

```
    for i in range(16):
        w.append(msg[16*i:16*i+16])
    for i in range(16,80):
        nw = ((s_512(w[i-2],1)+s_512(w[i-15],0)+Convert_hex_to_int(w[i-16])+Convert_hex_to_int(w[i-
7]))%(2**64))
        w.append(hex(nw)[2:])
    return w


def SHA_512(msg):
    [a,b,c,d,e,f,g,h]=state
    words = Generate_words(hex(msg)[2:].rjust(256,'0'))
    for r in range(80):
        t1 = (h+ch(e,f,g)+sigma(e,1)+Convert_hex_to_int(words[r])+K[r])%(2**64)
        t2 = (sigma(a,0)+maj(a,b,c))%(2**64)
        nh = g
        ng = f
        nf = e
        ne = (d+t1)%(2**64)
        nd = c
        nc = b
        nb = a
        na = (t1+t2)%(2**64)
        a,b,c,d,e,f,g,h = na,nb,nc,nd,ne,nf,ng,nh
    strout = "0x"
    for x in [a,b,c,d,e,f,g,h]:
        strout+=hex(x)[2:].rjust(16,'0')
    bufs = [a,b,c,d,e,f,g,h]
    strout = "0x"
    for i in range(8):
        state[i] = (state[i] + bufs[i])%(2**64)
        strout+=hex(state[i])[2:].rjust(16,'0')
```

```
h = plain.encode('ascii').hex()
l = 4*len(h)
print(f"Input bits: {l}")
h = (Convert_hex_to_int(h)<<1) + 1
nblocks, npadding = Number_of_blocks(l)
h = (h<<(npadding+128))+(l)
print(f"Number of blocks: {nblocks}\n")
blocks = []
for i in range(nblocks):
    blocks.append(Convert_hex_to_int(hex(h)[2+256*i:258+256*i].rjust(256,'0')))
for block in blocks: SHA_512(block)
strout = "0x"
for i in range(8):
    strout+=hex(state[i])[2:].rjust(16,'0')
print(f"Final Hash: {strout}\n")
hash=int(strout,16)
w=pow(s,-1,q)
u1=(hash*w)%q
u2=(r*w)%q
v=((pow(g,u1,p)*pow(y,u2,p))%p)%q
print(f"W: {w}\n\nU1: {u1}\n\nU2: {u2}\n\nV: {v}\n")
if(v==r): print("VERIFIED SIGNATURE")
else: print("INVALID SIGNATURE")
```

Output:

Test Case 1 (No error):

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11> python .\DSS_sender.py
Enter message: 23BCE1145
Input bits: 72
Number of blocks: 1

Final Hash: 0x43bb94b9d96ed8f072819ba2a1d54ff03b7a7ed095bcd9deb0ecb65a1bfff6025f23ca82263c6c9369a44ac2dab9ac42ff2ba1679cb45b7626e57f50c6e50cf8

P: 14568200776206202130171533965825970473288439467678218345979591716145503428974443800006780838629925294518085459711839455958775414163836748438240719417324578893691302419972577671210128380494778362393473165895524944550817063811777211205574833755870177374132878571211680891031389623604747354866844509002471437641 3

Q: 1291154111742040931036651106067091809758275023813

G: 16594619865401294898322720929512554954594355241001436480400721225281243894998112359396587096056983293182941047283472757917212118335500583530168828347005672905148209121564795928431240282651275913280571036731200579720074552126710386436086042306476676859390060740520375124455924381549080455929698967008439966301 3

Sender Private Key(X): 489186874997206511564679394113292114510210104600

Per-message random number(K): 239614957002695135918312939292744757123017776 04

Sender Public Key(Y): 40670764320084380133044226837240330784628910791134726574299963441693194552616469597474960472824019457004479030199326087141002919023763370246268273469837872657120581837959197489856897010955696062274110579641347212211564665826356006354529656690881348987727673303539386323986807199918835573895926714679182861195

R: 284896963900532200878769584355243631278098382 16

S: 873024872468265760345448243089978232291548632169
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11> python .\DSS_receiver.py
P: 14568200776206202130171533965825970473288439467678218345979591716145503428974443800006780838629925294518085459711839455958775414163836748438240719417324578893691302419972577671210128380494778362393473165895524944550817063811777211205574833755870177374132878571211680891031389623604747354866844509002471437641 3

Q: 1291154111742040931036651106067091809758275023813

G: 16594619865401294898322720929512554954594355241001436480400721225281243894998112359396587096056983293182941047283472757917212118335500583530168828347005672905148209121564795928431240282651275913280571036731200579720074552126710386436086042306476676859390060740520375124455924381549080455929698967008439966301 3

Sender Public Key(Y): 40670764320084380133044226837240330784628910791134726574299963441693194552616469597474960472824019457004479030199326087141002919023763370246268273469837872657120581837959197489856897010955696062274110579641347212211564665826356006354529656690881348987727673303539386323986807199918835573895926714679182861195

Simulate error? Y/N: N

Message (M): 23BCE1145

Received R: 284896963900532200878769584355243631278098382 16

Received S: 873024872468265760345448243089978232291548632169

Input bits: 72
Number of blocks: 1

Final Hash: 0x43bb94b9d96ed8f072819ba2a1d54ff03b7a7ed095bcd9deb0ecb65a1bfff6025f23ca82263c6c9369a44ac2dab9ac42ff2ba1679cb45b7626e57f50c6e50cf8

W: 915701589294077640284602937413876969602825723871

U1: 677529806658180587017258278729160732707197257262

U2: 477414381059172542054745851552237897059258978 28

V: 284896963900532200878769584355243631278098382 16

VERIFIED SIGNATURE
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11>
```

Test Case 2 (Error):

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11> python .\DSS_sender.py
Enter message: 23BCE1145
Input bits: 72
Number of blocks: 1

Final Hash: 0x43bb94b9d96ed8f072819ba2a1d54ff03b7a7ed095bcd9deb0ecb65a1bfff6025f23ca82263c6c9369a44ac2dab9ac42ff2ba1679cb45b7626e57f50c6e50cf8

P: 14319534833021729444752859388023591541710362538646776929330391191304417315406553458259867144199463842030219914345418006609811900250807329514791982102501638652074156047150640563363031024897082374679931914649782116104961956133674963611364287527509261120001526505784737908308669309427236183890092047939030964844 3

Q: 1309232265906031298472837120334910675125192604371

G: 92245916396730647214266707489872888919570519618959603545686708688376794881935114786266070055551198334740627193468746117874999232189379617121980915678297607466813197015495640017187729985855187481655170355046639579699630435375094569264153701203728994603794527872495584245911234841916879475086769665043781409469

Sender Private Key(X): 590119733998321399513311722030803463323505549853

Per-message random number(K): 130508890566254175854636062338493362357145199910 8

Sender Public Key(Y): 81558055337608582633937240366563841289523312318238659647041903699995543179812958572749274893858085995093346498129768163651283147686340484656893538410697581429991333371839688283419927177171638727505876856833796479408277092171758147309237224926293625776445944661828941541206585294798969304353222189441934102783 4

R: 12966280389993699349777126620921403809257486956 77

S: 66616257638116448349759587884756469637043924017 7
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11> python .\DSS_receiver.py
P: 14319534833021729444752859388023591541710362538646776929330391191304417315406553458259867144199463842030219914345418006609811900250807329514791982102501638652074156047150640563363031024897082374679931914649782116104961956133674963611364287527509261120001526505784737908308669309427236183890092047939030964844 3

Q: 1309232265906031298472837120334910675125192604371

G: 9224591639673064721426670748987288891957051961895960354568670868837679488193511478626607005555119833474062719346874611787499923218937961712198091567829760746681319701549564001718772998585518748165517035504663957969963043537509456926415370120372899460379452787249558424591123484191687947508676966504378140946 9

Sender Public Key(Y): 8155805533760858263393724036656384128952331231823865964704190369999554317981295857274927489385808599509334649812976816365128314768634048465689353841069758142999133337183968828341992717717163872750587685683379647940827709217175814730923722492629362577644594661828941541206585294798969304353222189441934102783 4

Simulate error? Y/N: Y

Message (M): 23BEE1145

Received R: 1296628038999369934977712662092140380925748695677

Received S: 6661625763811644834975958788475646963704392401 77

Input bits: 72
Number of blocks: 1

Final Hash: 0xd3da669c5f9f5fc5ea4e646cdaf5333a0186950ffe7fa2298e1e40c06c7d1aa50fc0615d284d0532e37bb0cd3ca5c7779e10198c43f0cd9209bd618435c968c9

W: 1028068338309762800995656484635456373773865470 26

U1: 7667019904080120873598463807767892633100742472 9

U2: 7027957490855810129134209512026418357191604177 66

V: 7034663617137389315046346671842849616411267987 1

INVALID SIGNATURE
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-11>
```